EE5203 L03 Study Sheet

Functions, Arrays, Strings, and Multiple C Files - Basri Erdogan

Essential question: How do you split one long `main()` into named functions across several files — and still prove, with compiler and debugger evidence, that the pipeline computes the same results?

What you should be able to do

- Read a declaration and state the return type, name, and parameters.
- Explain declare (`.h`), define (`.c`), and call — and which build stage checks each.
- Predict pass-by-value behavior: parameters are local copies.
- Pass an array **and its element count** to a function.
- Choose integer widths: `uint16_t` per sample, `uint32_t` for the sum.
- Build a valid C string: capacity, text length, and the `\0` terminator.
- Use `strcmp(a, b) == 0` for equality — never `==` on strings.

The function contract

```c
int clamp(int value, int low, int high);
```

Read aloud: "`clamp` receives three integers and returns one integer."

Step	File	Contains	Checked by
Declare	`sensor_math.h`	signature, no body	compiler (types at each call)
Define	`sensor_math.c`	the body, exactly once	linker (finds one definition)
Call	`main.c`	`clamp(raw, 0, 4095)`	return value replaces the call

`#include "sensor_math.h"` is preprocessor text substitution — both `.c` files then see the same declaration. `undefined reference` is a **linker** message: a declaration exists but no definition was found.

Pass by value

```c
void raise_to_minimum(int value) { if (value < 100) value = 100; }

int raw = 42;
raise_to_minimum(raw);   /* raw is still 42 */
```

Parameters are copies. To send one result back, `return` it.

Arrays as parameters

```c
uint16_t average_samples(uint16_t values[], int count);
```

- `values[]` receives access to the first element — **not** a copy, and the function cannot discover the length: the caller must pass `count`.
- Loop with `i < count`; return `0U` when `count == 0`.
- Sum into `uint32_t`: a `uint16_t` sum can overflow from the 17th maximal sample (`17 x 4095 > 65535`).

`#define` constants

```c
#define SAMPLE_COUNT 8
```

The preprocessor replaces the name with `8` before compilation. It is not a variable and occupies no memory — and the array size and loop bound can no longer disagree.

C strings

```c
char status[8] = "READY";   /* capacity 8, length 5, '\0' at index 5 */
```

Term	Meaning
Capacity	Array size — where you may write (includes the `\0`)
Length	Characters before the `\0` (`"HIGH"` has length 4, needs 5 bytes)
`\0`	Tells string code where the text stops

- `{'O','K'}` is **not** a C string — no terminator.
- Compare with `strcmp(command, "START") == 0` (`<string.h>`).
- A string-writing function receives the destination **capacity**, checks it before writing, writes `\0` explicitly, and reports success (`bool`).

The L03 pipeline

```
acquire_samples(samples, SAMPLE_COUNT);          /* hardware -> array  */
mean = average_samples(samples, SAMPLE_COUNT);   /* array    -> value  */
make_status(message, MESSAGE_CAPACITY, mean);    /* value    -> string */
```

`main()` keeps the short list of steps; the detail lives in the functions.

Common mistakes

- Declaration and definition types that do not match.
- A non-`void` function that can reach its end without `return`.
- Assuming an array parameter knows its own length.
- `uint16_t` accumulator for a sum of many `uint16_t` values.
- Writing text that fills the array but leaves no room for `\0`.
- Comparing strings with `==`.
- Reading only the last build message instead of the first.

Mastery self-check

☐ I can label return type, name, and parameters in any declaration.
☐ I can say which build stage catches a bad argument type vs. a missing definition.
☐ I can explain why `raise_to_minimum(raw)` leaves `raw` unchanged.
☐ I can state the valid indices when `count == 8`.
☐ I can justify `uint32_t` for the sum with a concrete overflow count.
☐ I can draw `"OK"` in memory, byte by byte, inside `char status[8]`.
☐ I can write a capacity check for `make_status` before it writes.

Five review questions

1. `uint16_t average_samples(uint16_t values[], int count);` — what does the caller receive, and what two things does the caller have to supply?
2. The build fails with `undefined reference to make_status`. Which file is missing what, and why did compilation succeed?
3. After `int x = 7; scale(x, 4);` the student expects `x == 28`. What actually happened, and what one-line change fixes the caller?
4. How many bytes does `"HIGH"` occupy as a C string, and what is stored in the final byte?
5. `make_status(text, 4, 3500U)` must select `"HIGH"`. What should the function do, and what should it return?

See the L03 worksheet for extended practice. Worked solutions are released after the practice window.